



Guardrails, and Making Standards Travel

Lab 2.b — making a written rule refuse to be violated



Two hours, hands-on

8 min	Recap + why	You have documents. Nothing enforces them
10 min	The map + warm-up	Five layers on a timeline; the five-minute guardrail
21 min	The hook, twice	A git hook you already run → the Claude hook it explains · table talk
19 min	Demo — build along	One guardrail, verified, in YOUR copy
31 min	Your rule	Pick a written rule → hook → CI gate; both checkpointed
10 min	Review	Compare picks, false positives, severity calls
8 min	Hand-off, wrap & retro	The sixth artifact; reflect; capture



The law already on the books

One sentence: the law is written, and there is no police force.

- **CLAUDE.md has conventions.** Typed errors from lib/errors.ts; the logger, never console.
- **ADR-0001 has a law.** Every response goes through respond.ts — the envelope.
- **.claude/rules/testing.md has rules.** Vitest, co-located tests, query by role — no test ids.
- **And nothing enforces any of them.** That is today.



The model layer cannot be the control.

Models are non-deterministic and persuadable. A rule in CLAUDE.md holds most of the time and fails exactly when someone asks for something reasonable-sounding that violates it. Three words for the rest of the block: a GUARDRAIL is any deterministic mechanism — permission rule, hook, CI check — that refuses a violation instead of describing one. A HOOK is an interception point before or after a consequential action. A POLICY is the deterministic rule evaluated against it.



Five layers of enforcement

Each layer catches a different audience — today the enforcing layers each get a short demo, told as a timeline: authoring → commit → merge.

Layer	Where it lives	Holds against
1 Instruction	CLAUDE.md, docs behind a trigger table	Nothing reliably — it informs
2 Claude hooks + permissions	.claude/settings.json — deterministic, client-side	The agent, at authoring time
3 Commit-time	git hooks, commit lint	Anyone committing locally
4 CI gates	workflows, static analysis, custom validators	Anyone merging — agent or human
5 Documented bypass	a written escape hatch with a named approver	Nothing — it keeps 1–4 honest

This repo ships guardrails only at layers 1–2 (its CI runs lint and tests, but no governance gate yet). Today you build in 1–2 and wire ONE layer-4 gate so you feel what it costs. Layer 3 you will watch get built — adding it to your copy is a stretch.



Where organizations start, and where this grows

Most organizations begin with almost none of this — and that is normal. One team grew from exactly there to four custom static-analysis rules and nine merge-blocking validators, one rule at a time.

Layer	A typical starting point	The aspiration
3 Commit-time	none, or one stale lint hook	commit lint, pre-commit checks on staged files
4 CI gates	build + unit tests only	custom static-analysis rules, merge-blocking validators
5 Documented bypass	unwritten — people just know who to ask	APPROVED- conventions + telemetry on fire/bypass rates

That team governs its own AI agent the same way — an ordinary NFR with MUST clauses and thresholds; rules about AI are just rules. Everything you build today is a first step on this road.



The five-minute guardrail

Three verdicts, not two: allow (silent), ask (stop and check), deny (never). The middle one is the one teams forget.

.claude/settings.json

```
{
  "permissions": {
    "allow": [],
    "ask": ["Bash(rm *)"],
    "deny": [
      "Edit(server/data/seed.json)",
      "Write(server/data/seed.json)",
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Bash(git clean *)",
      "Bash(npm publish *)",
      "Bash(curl * | bash)"
    ]
  }
}
```

MERGE, never paste: the first two deny entries are ALREADY in your file, shown so you see where yours go — and your hooks block must survive. The handout shows the before/after. Then: ask Claude to force-push, watch it refuse; confirm seed.json is still protected.



The hook you already know — watch one get built

Your own repos already run one of these. CLAUDE.md here says "Conventional Commits" — and nothing checks it. Five lines fix that — and every row is identical on macOS, Linux, and Windows. The walkthrough replays it in your copy.

#	What I do	What you should see
1	npm install -D husky, then npx husky init	.husky/ + a starter pre-commit (we delete it); "prepare": "husky" lands
2	cp docs/labs/snippets/commit-msg.sh .husky/commit-msg	Five lines land — same command on Windows (PowerShell aliases cp)
3	Nothing to make executable — on ANY OS	husky's shim runs your file via sh: no chmod, no extension, no shebang needed
4	git commit --allow-empty -m "changed stuff"	REFUSED — BLOCKED/Why/Instead/Done; the commit never happens
5	git commit --allow-empty -m "docs: explain the format"	Passes, silently — exit 0 is the 99% case
6	git commit --no-verify -m "changed stuff"	The built-in bypass — every layer has one; this one ships with git
7	(not done live) fresh clone, no npm install	No hooks — layer 3 travels with the clone; why merge time exists



Same anatomy, three moments in time

One anatomy: an event fires a script, and the exit code decides. You met it at commit time; the next demo moves it to authoring time.

	Git hook (commit)	Claude hook (authoring)	CI gate (merge)
Fires when	you commit	the agent acts — a tool call	a PR opens or updates
Input	the commit-message file	the tool input, JSON on stdin	the diff vs the base branch
Verdict	non-zero exit → no commit	exit 2 → blocked; stderr → agent	failed check → no merge
Holds against	anyone committing, wired clone	the agent, at authoring time	anyone merging — agent or human
The bypass	git commit --no-verify	edit settings.json	the documented escape hatch

You have already lived with one — `protect-seed.js` is a `PreToolUse` hook; it blocked you in Lab 1. Its four-part message (BLOCKED / Why / Instead / Done) is the house pattern.



Claude hooks — the sixty-second refresher

You did the Hooks lesson; here is the whole contract on one slide — and you will reuse this exact JSON shape in the exercise.

THE CONTRACT

- Fires on Claude Code events — today: PreToolUse (before the agent acts) and PostToolUse (just after).
- The matcher picks which tool calls it sees: Edit|Write.
- Your script reads the tool input as JSON on stdin.
- exit 0 allows · exit 2 blocks — and stderr goes to the agent.
- Hooks load at session start — register, then NEW session.

.CLAUDE/SETTINGS.JSON

```
"hooks": {  
  "PreToolUse": [{  
    "matcher": "Edit|Write",  
    "hooks": [{  
      "type": "command",  
      "command": "node \"$CLAUDE_PROJECT_DIR  
        /.claude/hooks/protect-seed.js\""  
    }]  
  }]  
}
```

// the hook that blocked you in Lab 1
// – you have already lived the contract



Block or nudge? Decide, do not default

The event you register on, plus the exit code, IS the severity decision.

Event	What exit 2 does	Use it for
PreToolUse	BLOCKS the call. stderr goes to Claude	Unrecoverable or unambiguous violations
PostToolUse	Cannot block — the tool already ran — but stderr still reaches Claude	Heuristic rules; things you should not interrupt
exit 0	Allows, silently	The 99% case
any other code	Non-blocking; the user sees a hook-error notice	Almost never on purpose

Three tags for any rule: MECHANICAL (a script can decide), PARTLY (a heuristic — false positives possible), FUZZY (human judgment). The middle case is exactly what nudges are for. Not every guardrail should block: a gate that fires on legitimate work teaches people to route around it.



What rule would you add next?

Four minutes at your table. Pick ONE rule you'd love never to have to raise in review again — then place it on the timeline: authoring, commit, or merge. Post one line per table in chat: the rule + the moment + block or nudge. On the call solo? Answer for the repo you ship from.



Demo — build along, one guardrail

The read-only database rule, in YOUR copy — the commit-msg hook, moved to authoring time.



Seven steps — build along on screen

Steps 6 and 7 are the ones that matter — an unverified guardrail is a belief. Full walkthrough with the code: [docs/labs/lab-2b-demo-3-claude-hook-walkthrough.md](#).

#	What we do	What you should see
1	npm run reset-db	db.json exists (it is generated and git-ignored)
2	Add the Don't line to CLAUDE.md	Layer 1: "db.json is generated — change data via the API or a reset"
3	Add the deny entries to settings.json	Layer 2a: Edit/Write(server/data/db.json) in the existing deny array
4	Write the hook: .claude/hooks/protect-db.js	Layer 2b: the 20 lines on the check slide — BLOCKED/Why/Instead/Done
5	Register it; start a NEW session	settings.json hooks block gains the entry; hooks load at session start
6	Verify the BLOCK: ask Claude to edit db.json	Deny answers first (wall) → pull the 2 deny entries → hook answers (signpost) → restore
7	Verify the ALLOW: edit a repositories/ file; npm test	No refusal, no noise, suite green — then document the bypass in CLAUDE.md



Read-only database unless approved

The same three-layer shape as the seed guard, on a new subject — built along with me.

- **First:** ``npm run reset-db`` — `db.json` is generated and git-ignored, so a fresh copy does not have it.
- **The rule.** ``db.json`` gets changed through the API or a reset — not edited directly.
- **Not the store-boundary rule.** That one (you may pick it later) is about **imports** of ``db/store.ts``; this is about **editing a file**. Same family, two rules.
- **Three layers, again.** A Don't line → a ``permissions.deny`` entry → a `PreToolUse` hook that explains itself.



Twenty lines, no dependencies

Node, cross-platform, no packages. Half this room is on Windows.

[.claude/hooks/protect-db.js](#)

```
// after the skeleton's stdin plumbing has parsed filePath:
const path = filePath.replaceAll('\\', '/');

if (path.endsWith('server/data/db.json')) {
  console.error([
    'BLOCKED: db.json is generated, not edited.',
    'Why: hand-edits are lost on the next reset and',
    '  are invisible to everyone else.',
    'Instead: change data through the API, or edit',
    '  seed.json via PR and run npm run reset-db.',
    'Done means: your data change survives a reset.',
  ].join('\n'));
  process.exit(2); // PreToolUse -> blocks
}
process.exit(0);
```

Registered on PreToolUse for Edit|Write — registration JSON is in the handout, and a NEW session is needed after registering: hooks load at session start.



Verify both directions, then the bypass

A guardrail fails two ways — never blocking (useless) or blocking real work (worse: it gets disabled by Friday). So you test BOTH directions — then write the escape hatch down.

STEP 6 · BLOCK THE BAD

- Ask for the violation on purpose
- Hear both layers — wall, then signpost
- Save the hook's text — your PR evidence

STEP 7 · ALLOW THE GOOD

- Ask for legitimate work: a repositories/ file
- Expect SILENCE — and npm test still green
- The half everyone skips — false positives kill layers

THEN · WRITE THE BYPASS DOWN

- Every rule needs a legal route for the blocked thing
- Here: seed.json via a flagged PR + reset-db
- Never "just disable the hook" — one line in CLAUDE.md



Checkpoint 1

NOBODY ADVANCES UNTIL THIS IS TRUE

In YOUR copy: Claude is refused when it edits `server/data/db.json`, and succeeds when it edits a file in `server/src/repositories/`.

Paste your refusal text into chat. If you only tested the block, you have not finished — run the allow case now.



Pick a rule — the law is already written

Pick ONE — or bring a rule of your own. Every row is already written in this repo and enforced by nothing.

The rule	Where it is written	Hook shape
A server/src change ships with a sibling test	CLAUDE.md · testing approach	PostToolUse nudge — edited *.ts has no sibling *.test.ts
Only repositories/ may import db/store	CLAUDE.md · Don't list	PreToolUse block — written content imports db/store
Throw typed errors, not new Error()	CLAUDE.md · code conventions	PostToolUse nudge — content has throw new Error(
Responses only via respond.ts	ADR-0001 + CLAUDE.md	PostToolUse nudge — route content has res.json / res.send

No wrong choice. The hook-shape column is a hint, not a spec — the skeleton has both shapes ready.



The hook — give your rule teeth

Fifteen minutes. The plumbing is given; the rule is your pick from the menu.

- **Start from the skeleton.** ``docs/labs/snippets/hook-skeleton.js`` → ``'.claude/hooks/check-convention.js``; delete the two example rules; put yours in.
- **Register it, then NEW session.** The settings JSON is in the handout — matcher, event, ``$CLAUDE_PROJECT_DIR``. Hooks load at session start.
- **Block or nudge — the event decides.** PreToolUse blocks; PostToolUse nudges. Path rules can block; content heuristics should nudge.
- **Verify both directions.** Violation on purpose → save the text. Legitimate edit → silence. The text goes in your PR.



Checkpoint 2

NOBODY ADVANCES UNTIL THIS IS TRUE

Your hook fires on the violation, stays silent on a legitimate edit, and the text is pasted in your PR description.

The hook's text is the evidence standard — "it worked" is not verification.



The same rule, written again — for GitHub

GitHub cannot run your Claude hook — no agent runs there. So you write the SAME check again, as a grep in a workflow GitHub runs on every pull request: everyone, every machine.

#	What you do	What you should see
1	Copy snippets/governance-gate.yml → .github/workflows/governance.yml	The skeleton: checkout, changed-files list, an example check
2	Re-write your hook's check as the grep in the marked step	Same rule as check-convention.js — a second program, in a second place
3	Commit, push — and open a DRAFT PR	The gate runs ONLY when a PR exists; a pushed branch runs nothing
4	Violate your rule in the PR	RED — the merge is stopped, for anyone: agent or human
5	Fix it; push again	GREEN — red-then-green is your Checkpoint 3 evidence

Keep the skeleton's fetch-depth: 0 — the default shallow clone breaks the diff against the base branch with an unhelpful error.



Checkpoint 3

NOBODY ADVANCES UNTIL THIS IS TRUE

Your gate fails a PR that violates your rule, and passes one that does not.

Paste your PR link in chat with the check visibly red then green. The run URL is this layer's evidence — the hook text's merge-time twin.



Let us compare

Ten minutes. Two or three volunteers share their screen — not everyone.

- Which rule did you pick — and why that one?
- Whose hook has a false positive — and how did you find out?
- Who chose nudge over block, and what was the reasoning?
- What does your gate do on a docs-only PR?
- Which rule would you take to YOUR repo tomorrow?



The sixth artifact, and it is yours

Five governance artifacts live inside a repo — CLAUDE.md, ADR, NFR, recipe, rules file. The sixth is the only Collective-layer one — how a standard reaches every repo — and we are deliberately not handing you the answer, because choosing the mechanism, the rung, and the owner IS the roll-up.

WHAT THE GUIDE GIVES YOU

- Four words that carry it: standard, drift, pin, enforce.
- CRAWL: one repo + two CLAUDE.md lines — a 4-step checklist, ~30 min.
- WALK: queryable (3 tool options, traps named) + pinned copies.
- RUN: gates, served docs, a versioned plugin — each needs an owner.
- And a fill-in template for recording what you chose.

THE DECISION TEMPLATE

```
# docs/best-practices/  
#   distributing-standards.md  
  
# what "done" looks like:  
Owner:      <a person, not a team>  
Rung:       crawl | walk | run  
First move: <the ~30-min step>  
Decided:    YYYY-MM-DD  
Next review: YYYY-MM-DD  
  
# ...plus: where standards live,  
# how a project gets them, current-  
# ness, enforced vs advisory,  
# bypass, NOT-doing-yet
```



What leaves this room

Questions & support: #extra-duty-solutions-ai-training — your instructor and the Applied AI Engineer are there.

- **Ship checklist.** Warm-up merged (seed rules intact) · a hook enforcing a written rule, verified both ways, its text in the PR · the same rule as a CI gate, seen red then green · PR open, CI green.
- **Reflection note, now.** 3–5 sentences in your PR: what you learned, what surprised you, what you'd apply tomorrow. Acceptance criterion, not homework.
- **One retro question.** What was hard, what was easy — one line each, in chat, waterfall style.
- **Three continuations.** Port one artifact in office hours (bring the repo, not a question) · define how standards travel (owner + template) · apply the pattern to a convention of your own (the takeaway card) — or wire the commit-msg hook into your copy (the demo-2 walkthrough has every step).